

Frontend Creational and Structural Design Patterns: One-Stop Reference

This guide covers all major creational and structural design patterns relevant to frontend engineering, with detailed code snippets and developer comments for each.

1. Creational Patterns

1.1 Singleton

Ensures a class has only one instance and provides a global point of access to it.

```
// Singleton pattern in JavaScript
class ThemeManager {
  constructor() {
    if (ThemeManager.instance) return ThemeManager.instance;
    this.theme = 'light';
    ThemeManager.instance = this;
  }
  setTheme(theme) { this.theme = theme; }
  getTheme() { return this.theme; }
}
// Usage
const theme1 = new ThemeManager();
const theme2 = new ThemeManager();
console.log(theme1 === theme2); // true
```

1.2 Factory

Creates objects without specifying the exact class of object to create.

```
// Factory pattern for UI components
function createButton(type) {
  if (type === 'primary') {
    return `<button class='btn btn-primary'>Primary</button>`;
  } else if (type === 'secondary') {
    return `<button class='btn btn-secondary'>Secondary</button>`;
  }
  return `<button class='btn'>Default</button>`;
}
// Usage
const btn = createButton('primary');
```

1.3 Abstract Factory

Creates families of related objects without specifying their concrete classes.

```
// Abstract Factory for themed widgets
function createWidgetFactory(theme) {
```

```

return {
  createButton: () => `<button class='btn btn-${theme}'>${theme} Button</button>`,
  createInput: () => `<input class='input input-${theme}' />`
};
}
// Usage
const darkFactory = createWidgetFactory('dark');
const btn = darkFactory.createButton();

```

1.4 Builder

Separates the construction of a complex object from its representation.

```

// Builder for query string
class QueryBuilder {
  constructor() { this.query = []; }
  add(key, value) { this.query.push(`${key}=${encodeURIComponent(value)}`); return this; }
  build() { return this.query.join('&'); }
}
// Usage
const qs = new QueryBuilder().add('page', 1).add('sort', 'asc').build();
// Output: 'page=1&sort=asc'

```

1.5 Prototype

Creates new objects by copying an existing object (prototype).

```

// Prototype pattern using Object.create
const buttonPrototype = {
  render() { return `<button>${this.label}</button>`; }
};
const okButton = Object.create(buttonPrototype);
okButton.label = 'OK';
const cancelButton = Object.create(buttonPrototype);
cancelButton.label = 'Cancel';
// Usage
okButton.render(); // <button>OK</button>

```

2. Structural Patterns

2.1 Adapter

Allows incompatible interfaces to work together.

```

// Adapter for different API response formats
function adaptUserResponse(apiUser) {
  // Converts snake_case to camelCase
  return {
    id: apiUser.id,
    userName: apiUser.user_name,

```

```

    email: apiUser.email_address
  };
}
// Usage
const apiUser = { id: 1, user_name: 'alice', email_address: 'alice@example.com' };
const user = adaptUserResponse(apiUser);

```

2.2 Decorator

Adds new functionality to an object dynamically.

```

// Decorator for logging function calls
function withLogging(fn) {
  return function(...args) {
    console.log(`Calling ${fn.name} with`, args);
    return fn(...args);
  };
}
// Usage
function add(a, b) { return a + b; }
const loggedAdd = withLogging(add);
loggedAdd(2, 3); // Logs call and returns 5

```

2.3 Facade

Provides a simplified interface to a complex subsystem.

```

// Facade for DOM manipulation
const DOMUtils = {
  get(id) { return document.getElementById(id); },
  setText(id, text) { this.get(id).textContent = text; },
  show(id) { this.get(id).style.display = 'block'; },
  hide(id) { this.get(id).style.display = 'none'; }
};
// Usage
DOMUtils.setText('status', 'Loading...');

```

2.4 Proxy

Provides a placeholder to control access to another object.

```

// Proxy for API rate limiting
function createApiProxy(apiFn, limit) {
  let calls = 0;
  return (...args) => {
    if (calls >= limit) throw new Error('Rate limit exceeded');
    calls++;
    return apiFn(...args);
  };
}

```

```
// Usage
const fetchData = () => fetch('/api/data');
const limitedFetch = createApiProxy(fetchData, 5);
```

2.5 Composite

Composes objects into tree structures to represent part-whole hierarchies.

```
// Composite pattern for UI tree
class UIComponent {
  constructor(name) { this.name = name; this.children = []; }
  add(child) { this.children.push(child); }
  render() {
    return `<div>${this.name}${this.children.map(c => c.render()).join('')}</div>`;
  }
}
// Usage
const root = new UIComponent('App');
const header = new UIComponent('Header');
const main = new UIComponent('Main');
root.add(header); root.add(main);
root.render();
```

2.6 Bridge

Decouples abstraction from implementation so the two can vary independently.

```
// Bridge for themeable UI components
class Button {
  constructor(theme) { this.theme = theme; }
  render() { return `<button class='btn btn-${this.theme.style}'>${this.theme.label}</button>`; }
}
class DarkTheme { constructor() { this.style = 'dark'; this.label = 'Dark'; } }
class LightTheme { constructor() { this.style = 'light'; this.label = 'Light'; } }
// Usage
const darkBtn = new Button(new DarkTheme());
darkBtn.render(); // <button class='btn btn-dark'>Dark</button>
```

2.7 Flyweight

Reduces memory usage by sharing as much data as possible with similar objects.

```
// Flyweight for icon rendering
class IconFactory {
  constructor() { this.icons = {}; }
  getIcon(type) {
    if (!this.icons[type]) {
      this.icons[type] = `<svg class='icon icon-${type}'></svg>`;
    }
  }
}
```

```
        return this.icons[type];
    }
}
// Usage
const factory = new IconFactory();
const icon1 = factory.getIcon('user');
const icon2 = factory.getIcon('user');
console.log(icon1 === icon2); // true
```

References

- <https://refactoring.guru/design-patterns>
- <https://www.patterns.dev/>
- [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Details of the Object Model](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Details_of_the_Object_Model)

Each pattern above includes a real-world frontend use case, code, and inline dev comments for clarity. For behavioral and functional patterns, see the companion file or request more examples.